Activity Sheet 4 Date: 28.02.2026

Department of Electronics and Communication Engineering

Title of Experiment

Logistic Regression for Smart IoT Classification

Name of the Student	RAGHU NANDHAN
Registration Number	CH.EN.U4CCE24026
Due date of submission:28.02.2026	Date of Submission: 28.02.2026

Assessment Rubrics

Description	Mark Allotted	Mark Secured
Aim, Software required and Important definitions	10	
Model Execution	40	
Experimental Result	30	
Viva questions	20	
Late Submission	10	
Total Marks	100	

Signature ofLab in-charge

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Title: Logistic Regression for Smart IoT Classification

Aim: Regression. ML Model Used: Logistic Regression

Tasks:

1. Convert sensor data into classification labels (e.g., normal/fault, ON/OFF).
2. Train a Logistic Regression model.
3. Evaluate using accuracy, confusion matrix, and precision-recall.
4. Interpret classification results.

IoT Use Case: Fault detection, intrusion detection, device state prediction.

1. Aim: To build a Logistic Regression model that classifies IoT sensor data into categories such as Normal/Fault or ON/OFF, and evaluate its performance using accuracy, confusion matrix, and precision-recall metrics.

2. Software Required: Jupyter Notebook

3. Program/Task:

Task 01: Convert sensor data into classification labels (e g., normal/fault, ON/OFF).

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification
from google.colab import files

uploaded = files.upload()

train_data = pd.read_csv("datatraining.txt")
test_data1 = pd.read_csv("datatest.txt")
test_data2 = pd.read_csv("datatest2.txt")

test_data = pd.concat([test_data1, test_data2])

train_data = train_data.drop(columns=["date"])
test_data = test_data.drop(columns=["date"])

X_train = train_data.drop(columns=["Occupancy"])
y_train = train_data["Occupancy"]

X_test = test_data.drop(columns=["Occupancy"])
y_test = test_data["Occupancy"]

print("Task 1 Result:")
```

Task 02: Train a Logistic Regression model.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)

print("Task 2 Result:")
print("Model Trained Successfully")
```

Task 03: Evaluate using accuracy, confusion matrix, and precision-recall.

```
y_pred_log = log_model.predict(X_test_scaled)
log_accuracy = accuracy_score(y_test, y_pred_log)

print("Task 3 Result:")
print("Accuracy:", round(log_accuracy,4))
print(confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))
```

Task 04: Interpret classification results.

```
print("Task 4 Result:")
print("Final Logistic Regression Accuracy:", round(log_accuracy,4))
```

```
Task 4 Result:
Final Logistic Regression Accuracy: 0.9834
```

4. Experimental Results:

TASK 1

```
Choose files  3 files
datatest.txt(text/plain) - 200766 bytes, last modified: 03/03/2026 - 100% done
datatest2.txt(text/plain) - 699664 bytes, last modified: 03/03/2026 - 100% done
datatraining.txt(text/plain) - 596674 bytes, last modified: 03/03/2026 - 100% done
Saving datatest.txt to datatest (1).txt
Saving datatest2.txt to datatest2 (1).txt
Saving datatraining.txt to datatraining (1).txt
Task 1 Result:
Occupancy
0    6414
1    1729
Name: count, dtype: int64
```

TASK 2

```
Task 2 Result:
Model Trained Successfully
```

TASK 3

```
Task 3 Result:
Accuracy: 0.9834
[[9290  106]
 [ 100 2921]]
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      9396
           1       0.96      0.97      0.97      3021

    accuracy                           0.98     12417
   macro avg       0.98      0.98      0.98     12417
weighted avg       0.98      0.98      0.98     12417
```

S

TASK 4

```
Task 4 Result:
Final Logistic Regression Accuracy: 0.9834
```

5. Inference:

The Logistic Regression model accurately classified the IoT occupancy data with high performance. The confusion matrix shows minimal misclassification. The model is efficient and suitable for real-time IoT event detection.

Viva Questions:

1. What is Logistic Regression?

   Logistic Regression is a supervised learning algorithm used for binary classification that predicts probabilities using a sigmoid function.

2. Difference between Linear Regression and Logistic Regression.
   Linear Regression predicts continuous values, while Logistic Regression predicts probabilities for classification problems.

3. What type of problems can Logistic Regression solve?
   It solves binary and multi-class classification problems such as Yes/No or Normal/Fault prediction.

4. What is a sigmoid function?
   A sigmoid function maps any real value into a range between 0 and 1, producing probability outputs.

5. What is a confusion matrix?
   A confusion matrix is a table that shows correct and incorrect predictions made by a classification model.

6. Define accuracy, precision, recall.
   Accuracy is the overall correctness of the model, precision measures correct positive predictions, and recall measures how many actual positives were correctly identified.

7. Give an IoT use case for binary classification.
   Occupancy detection in smart buildings (Occupied/Not Occupied) is a common IoT binary classification use case.

8. Why is Logistic Regression suitable for event detection in IoT?
   It is computationally efficient, provides probability outputs, and works well for real-time binary event detection.

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Activity Sheet 5 Date:

Department of Electronics and Communication Engineering

Title of Experiment

K-Nearest Neighbors (KNN) for IoT Pattern Recognition

Name of the Student	RAGHU NANDHAN
Registration Number	CH.EN.U4CCE24026
Due date of submission: 28.02.2026	Date of Submission: 28.02.2026

Assessment Rubrics

Description	Mark Allotted	Mark Secured
Aim and Software required	10	
Model Execution	40	
Experimental Results	30	
Viva questions	20	
Late Submission	10	
Total Marks	100	

Signature ofLab in-charge

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Title: K-Nearest Neighbors (KNN) for IoT Pattern Recognition

Aim: To implement KNN for classifying IoT sensor data.

ML Model Used: K-Nearest Neighbors (KNN)

Tasks:

1. Choose an IoT classification problem (occupancy or activity detection).
2. Experiment with different K values.
3. Analyze accuracy vs K.
4. Compare KNN with Logistic Regression results.

IoT Use Case: Occupancy detection, smart home activity recognition.

1. Aim: To implement the K-Nearest Neighbors (KNN) algorithm to classify IoT sensor data and analyze its performance by evaluating accuracy for different K values.

2.Software Required: : Jupyter Notebook

3.Program/Task:

Task 01: Choose an IoT classification problem (occupancy or activity detection).

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from google.colab import files

uploaded = files.upload()

train_data = pd.read_csv("datatraining.txt")
test_data1 = pd.read_csv("datatest.txt")
test_data2 = pd.read_csv("datatest2.txt")

test_data = pd.concat([test_data1, test_data2])

train_data = train_data.drop(columns=["date"])
test_data = test_data.drop(columns=["date"])

X_train = train_data.drop(columns=["Occupancy"])
y_train = train_data["Occupancy"]

X_test = test_data.drop(columns=["Occupancy"])
y_test = test_data["Occupancy"]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Task 1 Result:")
print("Occupancy Detection Selected")
```

Task 02: Experiment with different K values.

```
k_values = [1,3,5,7,9]
knn_accuracies = []

for k in k_values:
    knn_model = KNeighborsClassifier(n_neighbors=k)
    knn_model.fit(X_train_scaled, y_train)
    y_pred_knn = knn_model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred_knn)
    knn_accuracies.append(acc)
    print("K =", k, "Accuracy =", round(acc,4))
```

Task 03: Analyze accuracy vs K.

```
plt.plot(k_values, knn_accuracies, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("Accuracy vs K")
plt.show()

print("Accuracy Values:", knn_accuracies)
```

Task 04: Compare KNN with Logistic Regression results.

```
best_knn = max(knn_accuracies)

print("Task 4 Result:")
print("Best KNN Accuracy:", round(best_knn,4))
```

2. Experimental Results:

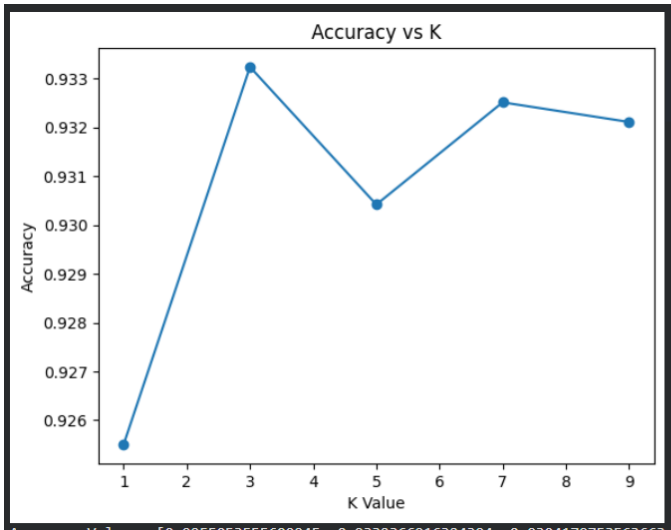
TASK 1

```
Choose files 3 files
datatest.txt(text/plain) - 200766 bytes, last modified: 03/03/2026 - 100% done
datatest2.txt(text/plain) - 699664 bytes, last modified: 03/03/2026 - 100% done
datatraining.txt(text/plain) - 596674 bytes, last modified: 03/03/2026 - 100% done
Saving datatest.txt to datatest (2).txt
Saving datatest2.txt to datatest2 (2).txt
Saving datatraining.txt to datatraining (2).txt
Task 1 Result:
Occupancy Detection Selected
```

TASK 2

```
K = 1 Accuracy = 0.9255
K = 3 Accuracy = 0.9332
K = 5 Accuracy = 0.9304
K = 7 Accuracy = 0.9325
K = 9 Accuracy = 0.9321
```

TASK 3



colab.research.google.com – to exit full s

```
Accuracy Values: [0.9255053555609245, 0.9332366916324394, 0.9304179753563663, 0.9325118788757348, 0.9321092051220101]
```

TASK 4

```
Task 4 Result:
Best KNN Accuracy: 0.9332
```

Inference

The KNN model successfully classified the IoT occupancy data, and accuracy varied with different K values. The model performed best at an optimal K, showing that proper K selection is important for better prediction performance.:

Viva Questions:

1. What is the KNN algorithm?
   KNN is a supervised learning algorithm that classifies data based on the majority class among its K nearest neighbors.

2. How does KNN classify a data point?
   It calculates the distance between the new data point and training points, then assigns the class most common among the nearest K neighbors.

3. What is the role of the K value?
   The K value determines how many neighbors are considered when making a classification decision.

4. What happens if K is very small or very large?
   A very small K can cause overfitting, while a very large K can lead to underfitting anD reduced model accuracy.

5. Why is data scaling important for KNN?
   Scaling ensures that all features contribute equally to distance calculation, preventing bias toward larger-valued features.

6. Which distance metric is commonly used in KNN?
   Euclidean distance is the most commonly used distance metric in KNN.

7. Give an IoT application of KNN.
   KNN can be used for smart home occupancy detection based on sensor data.

8. What are the disadvantages of KNN for large IoT datasets?

   KNN is computationally expensive and slow for large datasets because it stores all training data and calculates distances for each prediction.